

# SC6103 Distributed Flight Information System

Course project report | 10 October 2026 | Protocol v1.0

**Team review copy. Implementation and local integration are complete. Actual contribution percentages and the three-computer demonstration record require team completion before official submission.**

| Member       | Responsibility                                                             | Actual contribution     |
|--------------|----------------------------------------------------------------------------|-------------------------|
| Zhang Zhiyin | Part A: protocol, UDP server, invocation semantics and integration         | To be confirmed by team |
| Peng Jinyu   | Part B: flight business logic, monitor service and experiment design       | To be confirmed by team |
| Ji Chengyu   | Part C: Python client, invocation retries, monitor reception and interface | To be confirmed by team |

The final percentages must sum to 100%. Planned task weights are not measurements of actual contribution. This report records implemented behavior and observed results, without claiming that local tests constitute a demonstration on three physical computers.

# 1. Objectives and architecture

The system provides flight information, reservations and time-limited seat monitoring over IPv4 UDP. It also implements an idempotent fare-setting operation and a non-idempotent fare-increase operation. A Java server interoperates with Python clients through an explicit binary protocol. The production system uses only the JDK and Python standard library; it does not use RMI, RPC frameworks, CORBA, Java object serialization or stream-based network marshalling.

The client menu and experiment driver call the same Invoker. The Invoker allocates a session/request identity, encodes the request once and uses one UDP socket for requests, replies and callbacks. The server receives a datagram, validates it, applies the selected invocation semantics and delegates business work to DefaultFlightService. That service shares one InMemoryMonitorService with the dispatcher. Business code returns typed responses and callback events rather than sending packets itself.

The server uses a sequential receive/dispatch loop. This makes history lookup, business execution and result recording ordered without a thread per request. A 250 ms receive timeout permits subscription cleanup while idle. The shutdown hook closes the socket to unblock reception. Business data, subscriptions and duplicate history are in memory and reset on restart.

Part A implements BinaryProtocolCodec, UdpTransport, UdpServer, RequestDispatcher, InMemoryRequestHistory, LossSimulator and structured ServerLog output. Existing Part B business logic and Part C client implementation remain the shared integration baseline. ProtocolException gained local error-reply metadata while retaining its existing constructor; this does not change wire fields or shared DTOs.

## 2. Explicit binary protocol

All multi-byte numbers use network byte order. Signed integers occupy four bytes; fares use IEEE 754 binary32. Locations and error text use strict UTF-8, prefixed by a four-byte byte count rather than a character count. Java manually shifts integer/UUID bytes and converts float bits; it does not serialize objects or use input/output streams to marshal messages. Python follows the same field layout. Dates contain five integers: year, month, day, hour and minute, interpreted in UTC+8.

| Header field | Bytes | Meaning                                |
|--------------|-------|----------------------------------------|
| version      | 1     | Protocol version 1                     |
| messageType  | 1     | Request 1, reply 2, callback 3         |
| operation    | 2     | Operations 1 through 6                 |
| sessionId    | 16    | Nonzero UUID                           |
| requestId    | 4     | Positive signed integer                |
| status       | 2     | Success 0 or defined error 1 through 9 |
| semantics    | 1     | ALO 1 or AMO 2                         |
| reserved     | 1     | Must be zero                           |
| bodyLength   | 4     | Exact bytes following the header       |

The header is exactly 32 bytes. The application maximum is 1024 bytes, while reception uses a 65535-byte buffer to detect oversized UDP messages rather than silently truncating them to the application limit. Decoding uses the actual packet offset and length. It rejects missing fields, extra bytes, negative lengths, malformed UTF-8, invalid response dates and non-finite response fares. Location text is limited to 128 encoded bytes and error text to 256 bytes. Typed but business-invalid request values are passed to business validation and receive a normal business error.

Malformed packets with a trustworthy request identity receive a protocol error; identities that cannot safely be recovered are silently dropped. Oversized messages return status 9 when identity is trustworthy. Unsupported operations return 6; mode mismatch returns 7 and echoes the requested mode so the client can match the error. Malformed-message status is 5 and conflicting reuse of an AMO request key is 8. These pre-business failures do not populate business history.

The normative byte layout, status definitions and fixed vectors remain in SC6103\_接口与数据类型规范.md. Java-to-Python and Python-to-Java vector tests cover requests, successful responses, errors and callbacks, including Chinese UTF-8 locations.

### 3. Flight operations and business state

| Operation            | Request body                   | Successful response body                            |
|----------------------|--------------------------------|-----------------------------------------------------|
| QUERY_ROUTE (1)      | Source and destination strings | Count followed by all matching flight IDs           |
| QUERY_FLIGHT (2)     | Flight ID                      | Five departure-time integers, fare, available seats |
| RESERVE_SEATS (3)    | Flight ID, quantity            | Flight ID, remaining seats                          |
| MONITOR (4)          | Flight ID, duration in seconds | Flight ID, remaining milliseconds                   |
| SET_AIRFARE (5)      | Flight ID, new fare            | Flight ID, resulting fare                           |
| INCREASE_AIRFARE (6) | Flight ID, positive increment  | Flight ID, resulting fare                           |

Route results are sorted and include all matches. Detail responses deliberately do not repeat the flight ID: clients already have the request context, and both implementations follow the agreed response shape. Reservation validation completes before state changes; invalid quantities, unknown flights and insufficient seats produce errors with no seat or sequence-number side effects.

The six-flight seed includes two Singapore-to-Beijing matches, a sold-out Singapore-to-Shanghai flight, Chinese-location routes and a reverse route. Flight 1001 starts with 10 seats, fare 100 and update sequence 0. Each experiment starts a fresh server and session to restore this known state. The controlled reservation quantity is 1, SET fare is 120 and INCREASE delta is 20.

SET\_AIRFARE is idempotent with respect to fare state: applying the same assignment twice leaves the same value. INCREASE\_AIRFARE is non-idempotent: two successful additions of 20 change 100 to 140. Reserving seats is also non-idempotent. Business validation rejects non-finite, out-of-range and ineffective floating-point changes as required by the shared specification. Fare changes do not generate seat-change callbacks.

## 4. Retries and invocation semantics

A client creates a fresh UUID session and increasing positive request IDs. A logical call is encoded once. Every retry retains identical bytes, UUID, request ID and source socket. Defaults are a 1000 ms timeout and five total attempts, including the initial send. Irrelevant, malformed or late messages do not extend an attempt's absolute deadline. Replies must match the configured server endpoint and outstanding request identity, operation and semantics.

In ALO mode, the server executes every valid received request, including duplicates. Retries improve the chance of completion, but a finite retry budget does not guarantee delivery when all requests are lost. A lost reply may cause a non-idempotent operation to execute multiple times.

In AMO mode, the server first checks history keyed by (session UUID, request ID). An entry retains the original request bytes, peer endpoint and immutable result. Matching duplicates replay the result without invoking the business method again. Reuse with different bytes or a different peer is rejected. Successes and business errors are saved before ordinary reply transmission or simulated reply loss. Thus a retry after a lost business error receives the same error even if other clients subsequently change the state.

The ordering is: validate trustworthy header and mode; check AMO history; decode a new request; execute business logic; encode and save the result; drop or send the ordinary reply; attempt each eligible callback. History has no eviction during a server process lifetime. ALO bypasses duplicate suppression. Both modes use the same business implementation.

AMO is an at-most-once execution guarantee within the retained in-memory history, not an exactly-once or crash-recovery guarantee. Restart discards history. If attempts expire, the client reports that confirmation was not obtained and the execution result is unknown. It does not automatically switch to a new identity and repeat a reservation or fare increase. A separate query can inspect current state but is not a transaction receipt under concurrent clients.

## 5. Time-limited server callbacks

MONITOR registers a client for a flight for 1 to 3600 seconds. Multiple clients may monitor the same flight; monitoring one flight does not subscribe to another. Successful reservations create events containing flight ID, remaining seats and increasing update sequence. The dispatcher sends a 44-byte callback from the same server socket directly to each active subscriber. Callback identity is the original registration UUID/request ID, not the reservation identity.

An ordinary reply being lost or its send raising an I/O exception does not suppress the callbacks produced by that reservation. One callback recipient failing does not stop attempts to the others. The server checks subscription activity immediately before each send and periodically purges expired registrations during idle reception. AMO replay never emits a second copy of the business event.

A monitor confirmation is the sole dynamic replay exception. Immediately before initial or duplicate confirmation transmission, the dispatcher reads the original registration's remaining lifetime. It does not re-register or extend that subscription. If the registration expired or was replaced, remainingMillis is zero. ALO duplicates do re-execute registration, consistent with that mode.

The client begins its local monitoring deadline from a successful confirmation's remainingMillis. Duplicate confirmations do not renew it. Before confirmation, only the highest-sequence callback for the pending registration is retained; after confirmation, only strictly increasing sequences from the expected server and registration are shown. Gaps are allowed because callbacks have no acknowledgement or retransmission. The menu resumes when monitoring ends. These callbacks provide best-effort updates, not a reliable event stream.

## 6. Fault experiments and observed results

All results below were observed with independent Java and Python processes using real loopback UDP on one computer. They are not simulated business outcomes or three-computer observations. Each case resets the server, uses a fresh session and records command lines plus client/server/proxy JSON Lines. Client request loss suppresses the selected initial request; server reply loss suppresses the selected initial ordinary reply after history insertion. The existing UDP proxy implements complete request or reply loss.

Initial state is 10 seats and fare 100. Each row below represents both an ALO and an AMO run: eight cases, sixteen runs. Execution/cache counts refer to the target mutation, not readiness or final-state queries. E means business executions; H means cache hits.

| Case                       | ALO observation             | AMO observation             |
|----------------------------|-----------------------------|-----------------------------|
| Baseline, no loss          | Seats 9, fare 140           | Seats 9, fare 140           |
| First reserve request lost | Seats 9; E=1, H=0           | Seats 9; E=1, H=0           |
| First reserve reply lost   | Seats 8; E=2, H=0           | Seats 9; E=1, H=1           |
| First increase reply lost  | Fare 140; E=2, H=0          | Fare 120; E=1, H=1          |
| First set-fare reply lost  | Fare 120; E=2, H=0          | Fare 120; E=1, H=1          |
| All reserve requests lost  | Seats 10; E=0, H=0; unknown | Seats 10; E=0, H=0; unknown |
| All reserve replies lost   | Seats 5; E=5, H=0; unknown  | Seats 9; E=1, H=4; unknown  |
| Quiet monitor expiry       | Completed at deadline       | Completed at deadline       |

Each first-loss mutation takes two attempts. Complete-loss cases use all five attempts and report unknown. Baseline successfully exercises all six operations. Quiet monitoring uses a two-second registration and waits until its deadline.

The SET experiment demonstrates why final state alone cannot establish AMO: both modes finish at 120, but execution logs show two business executions in ALO and one in AMO. Conversely, the two complete-loss cases are indistinguishable to the timing-out caller while their server-side outcomes differ. These results support the implementation's stated guarantees, without claiming statistical performance estimates from a single controlled run per case.

The analyzer correlates exact request identities and encoded bytes with BUSINESS\_EXECUTED, HISTORY\_SAVED, CACHE\_HIT and DROP\_REPLY events. It checks expected ordering and counts; it does not infer execution counts from attempts alone. A previously identified monitor-evidence weakness (R2) was fixed: an end timestamp must be finite and cannot precede the start or monitoring deadline. Regression tests reject prematurely ended synthetic logs.

## 7. Verification and reproducibility

The complete final validation ran from a source ZIP extracted into a clean directory without Git metadata or compiled classes, on macOS ARM64 with Python 3.13.1 and Temurin Java 17.0.20.1. Compilation used `javac 21.0.5` with `--release 17`. This is both a Java 17 target build and execution on an actual Java 17 runtime. Windows-specific PowerShell execution and Python 3.10 runtime execution were not part of this run.

| Verification suite                        | Passed groups/tests |
|-------------------------------------------|---------------------|
| A binary protocol                         | 19                  |
| A invocation semantics and transport      | 21                  |
| B business logic                          | 34                  |
| B monitor service                         | 15                  |
| C client                                  | 63                  |
| Experiment tools and evidence regressions | 36                  |
| Real Java/Python boundary integration     | 12                  |
| Total automated tests                     | 200                 |
| Additional real UDP experiment runs       | 16                  |

Boundary integration covers malformed/truncated/oversized packets, mode mismatch, UTF-8, duplicate identity conflicts, business-error replay, separate sessions, dual monitoring, flight isolation, expired/replaced registrations, and callbacks received before a retried registration confirmation. Deterministic unit tests inject reply/callback send failures and verify that other callback deliveries are still attempted. These injected failures are distinguished from actual-network tests.

The final evidence directory is `evidence/a/release-java17/`. `summary.json` records every suite's command and exit code, `environment.json` records versions, and `source-manifest.json` hashes the exact tested Java/Python sources. `matrix/results.csv` contains all sixteen actual results; per-case folders contain original logs and the server audit. `network/` contains boundary-test process evidence. Older development runs remain separately labelled.

From the repository root, with a JDK and Python on PATH:

```
python3 -m experiments.verify_all --matrix
```

This recompiles sources, runs all suites and writes a fresh evidence folder without overwriting the retained run. For ordinary use, compile with `sh server/build.sh` on macOS/Linux or `server/build.ps1` on Windows, then start `java -cp server/build/classes flight.ServerMain --bind 0.0.0.0 --port 6789 --semantics amo`. Run `python3 -m client --server 127.0.0.1 --port 6789 --semantics amo` in another terminal. Windows may use `python` instead of `python3`. Use the server's real IPv4 on other computers. The `--check` option performs offline configuration checks only.

## 8. Demonstration, limitations and completion

The prescribed three-computer demonstration remains pending. The proposed roles are Zhang Zhiyin on the server, Peng Jinyu on the monitor/query client, and Ji Chengyu on the reservation/second-monitor client. All machines use compatible semantics and independently generated sessions.

experiments/demo\_steps.md provides the sequence: six operations and errors; one monitor and one writer; two monitors; cross-flight isolation; expiry; and controlled ALO/AMO reply-loss comparisons. A second process on a client machine can issue a reservation while both interactive clients are monitoring.

The team must record actual IP addresses, OS/runtime versions, commands and logs on three different physical computers. Local integration evidence cannot establish routing, firewall or cross-machine behavior. The report cover also requires the team's agreed actual contribution percentages before submission.

The implemented system meets the functional local-integration scope: six operations, explicit cross-language wire encoding, server-driven monitoring, bounded retries, ALO/AMO selection, duplicate suppression and reproducible fault experiments. Remaining operational limits are intentional coursework scope: in-memory state and unbounded lifetime history, no authentication, no callback retransmission, and no crash-persistent transaction record. No claim of production reliability or exactly-once behavior is made.

## References and repository artifacts

1. SC6103 course assignment, Course.Project.for2026a.pdf, supplied course requirements (four pages).
2. Repository protocol specification: SC6103\_接口与数据类型规范.md, v1.0; field layouts, validation and reference vectors.
3. experiments/seed\_manifest.md and experiments/cases.md; seed data and controlled fault design.
4. evidence/a/release-java17/; actual environment, source hashes, test output, network logs and sixteen-run results.
5. server/A\_HANDOFF.md and report/A\_通信与调用语义.md; Part A implementation and integration handoff. Existing Part B and Part C chapter files retain their dated development records.
6. Repository: <https://github.com/Stevenhh7/SC6103-Distributed-System-Coursework>